Projects

- By Aditya Varma

Index

Movie Recommendation System in C

Problem Statement:

The objective of this project is to design and implement a **Movie Recommendation System** using the C programming language. The system helps users discover movies and web series from a collection of **Bollywood and Hollywood content**. Users can browse all available content, search by title or genre, view top-rated movies, sort content based on ratings, and receive personalized recommendations.

This project demonstrates the use of **arrays, structures, strings, functions, loops, conditionals, searching techniques, sorting algorithms, and menu-driven programming**.

Concepts Used:

- **Structures:** To store movie, series information such as title, genre, language, type, and rating.
- **Arrays of Structures:** To maintain a collection of movies and web series.
- **Functions:** To modularize different operations like searching, sorting, displaying, and recommending content.
- **Loops:** To traverse and process the movie database.
- **Conditional Statements (if-else):** To filter content and implement recommendation logic.
- **Switch Case:** To handle menu-driven user choices.
- **Strings (string.h):** To compare and manage movie titles, genres, and languages.
- **Searching:** To locate content by title or genre.
- **Sorting:** To arrange content based on ratings.
- **Formatted Input/Output:** To display content in a clean tabular format.

Short Description of Working:

1. The program starts by loading a predefined database of movies and web series.
2. A menu is displayed to the user with multiple options.
3. The user can:
    - View all available content.
    - Search content by title.
    - Search content by genre.
    - View top-rated movies and series.
    - Get personalized recommendations.
    - Sort content by ratings.
    - Update ratings for movies or series.
4. The search feature scans the database and displays matching content.
5. The recommendation feature suggests content based on language and rating.

6. The sorting feature arranges content from highest rated to lowest rated.
7. The rating feature allows users to update ratings of existing content.
8. The program continues to run until the user selects the Exit option.

Features:

📽️ **Content Library**
- Bollywood Movies
- Bollywood Series
- Hollywood Movies
- Hollywood Series

🔍 **Search Features**
- Search by Title
- Search by Genre

⭐ **Rating Features**
- Show Top Rated Content
- Update Ratings
- Sort Content by Rating

🎯 **Recommendation Features**
- Recommend Hindi Content
- Recommend English Content
- Suggest Highly Rated Movies and Series

📋 **Utility Features**
- View Complete Database
- Menu Driven Interface
- Easy Navigation

movieRecomendationSystem.c:

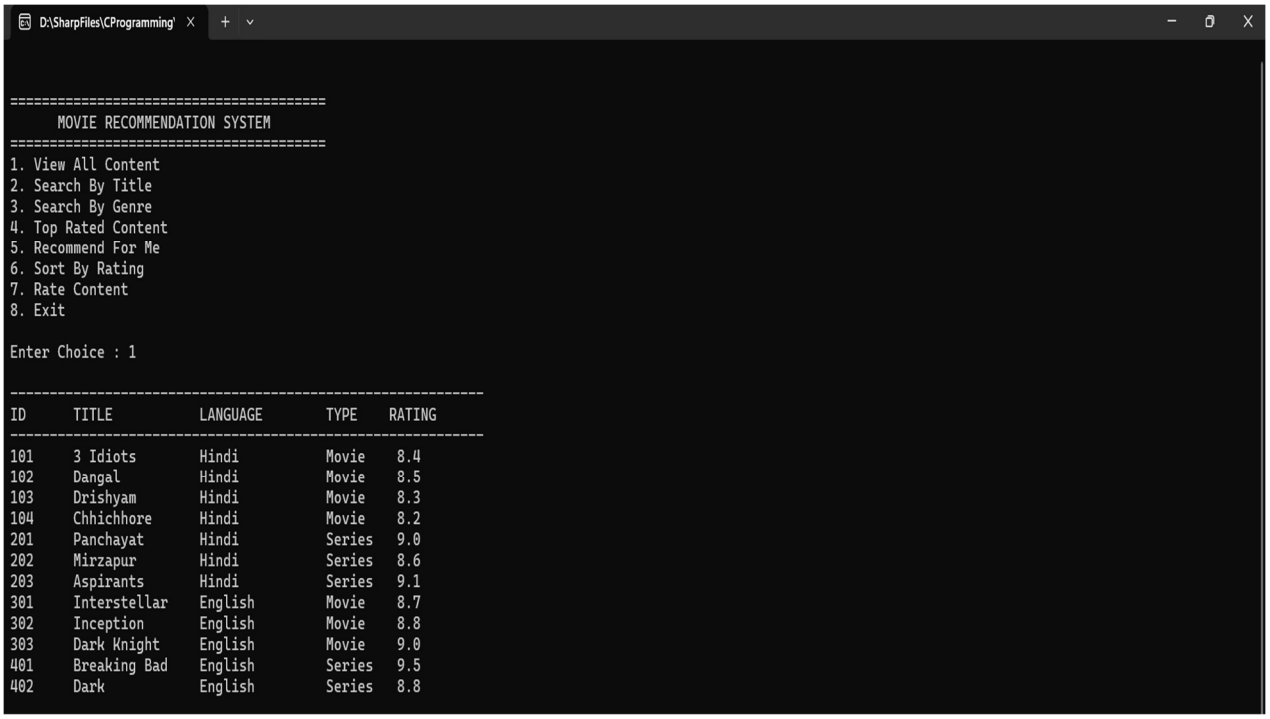
```c
/*-----------------------------------------------------------
    Project Title : Movie Recommendation System
    Description : A console based movie and series recommendation system
                  that allows users to:
        1. View all content
        2. Search by title
        3. Search by genre
        4. Show top rated content
        5. Recommend content
        6. Sort by rating
        7. Exit
    Concepts Used : Structures, Arrays, Strings, Functions,
                    Loops, Conditionals, Searching, Sorting
-----------------------------------------------------------*/

#include <stdio.h>
#include <string.h>

struct Movie {
    int id;
    char title[50];
    char genre[20];
    char language[20];
    char type[20];
    float rating;
};

struct Movie m[] = {
    {101, "3 Idiots", "Comedy", "Hindi", "Movie", 8.4},
    {102, "Dangal", "Drama", "Hindi", "Movie", 8.5},
    {103, "Drishyam", "Thriller", "Hindi", "Movie", 8.3},
    {104, "Chhichhore", "Comedy", "Hindi", "Movie", 8.2},
    {201, "Panchayat", "Drama", "Hindi", "Series", 9.0},
    {202, "Mirzapur", "Crime", "Hindi", "Series", 8.6},
    {203, "Aspirants", "Drama", "Hindi", "Series", 9.1},
    {301, "Interstellar", "SciFi", "English", "Movie", 8.7},
    {302, "Inception", "SciFi", "English", "Movie", 8.8},
    {303, "Dark Knight", "Action", "English", "Movie", 9.0},
    {401, "Breaking Bad", "Crime", "English", "Series", 9.5},
    {402, "Dark", "SciFi", "English", "Series", 8.8}
};

int n = sizeof(m) / sizeof(m[0]);

/*-----------------------------------------------
    Function : displayAll
    Purpose  : Display all movies and series
-----------------------------------------------*/
void displayAll() {
    int i;

    printf("\n-----------------------------------------------------------------\n");
    printf("ID\tTITLE\t\tLANGUAGE\tTYPE\tRATING\n");
    printf("-----------------------------------------------------------------\n");

    for(i = 0; i < n; i++) {
        printf("%d\t%-15s\t%-10s\t%-8s %.1f\n", m[i].id, m[i].title,
               m[i].language, m[i].type, m[i].rating);
    }
}

```

```c
/*-----------------------------------------------------
    Function : searchTitle
    Purpose  : Search content by title
-----------------------------------------------------*/
void searchTitle() {
    char name[50];
    int i, found = 0;

    printf("Enter Title : ");
    scanf(" %[^\n]", name);

    for(i = 0; i < n; i++) {
        if(strcmp(name, m[i].title) == 0) {
            printf("\nFound!\n");
            printf("Title    : %s\n", m[i].title);
            printf("Genre    : %s\n", m[i].genre);
            printf("Language : %s\n", m[i].language);
            printf("Type     : %s\n", m[i].type);
            printf("Rating   : %.1f\n", m[i].rating);

            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Content Not Found!\n");
}

/*-----------------------------------------------------
    Function : searchGenre
    Purpose  : Search by genre
-----------------------------------------------------*/
void searchGenre() {
    char genre[20];
    int i;

    printf("Enter Genre : ");
    scanf("%s", genre);

    printf("\nRecommendations:\n");

    for(i = 0; i < n; i++) {
        if(strcmp(genre, m[i].genre) == 0) {
            printf("- %s\n", m[i].title);
        }
    }
}

/*-----------------------------------------------------
    Function : topRated
    Purpose  : Show top rated content
-----------------------------------------------------*/
void topRated() {
    int i;

    printf("\nTop Rated Content\n");

    for(i = 0; i < n; i++) {
        if(m[i].rating >= 8.8) {
            printf("%s (%.1f)\n", m[i].title, m[i].rating);
        }
    }
}
```

```c
/*-----------------------------------------------------
    Function : recommendMovie
    Purpose  : Recommend based on language
-----------------------------------------------------*/
void recommendMovie() {
    int choice,i;

    printf("\n1. Hindi\n");
    printf("2. English\n");

    printf("Choose Language : ");
    scanf("%d", &choice);

    printf("\nRecommended For You\n");

    for(i = 0; i < n; i++) {
        if(choice == 1 && strcmp(m[i].language, "Hindi") == 0 && m[i].rating >= 8.5) {
            printf("%s\n",m[i].title);
        }

        if(choice==2 && strcmp(m[i].language, "English") == 0 && m[i].rating >= 8.5) {
            printf("%s\n", m[i].title);
        }
    }
}

/*-----------------------------------------------------
    Function : sortRating
    Purpose  : Sort content by rating
-----------------------------------------------------*/
void sortRating() {
    int i, j;
    struct Movie temp;

    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(m[j].rating < m[j+1].rating) {
                temp = m[j];
                m[j] = m[j+1];
                m[j+1] = temp;
            }
        }
    }

    printf("\nSorted By Rating (High to Low)\n");

    for(i = 0; i < n; i++) {
        printf("%s (%.1f)\n", m[i].title, m[i].rating);
    }
}

/*-----------------------------------------------------
    Function : rateMovie
    Purpose  : Update movie rating
-----------------------------------------------------*/
void rateMovie() {
    int id, i;
    float rating;

    printf("Enter Content ID : ");
    scanf("%d", &id);

    for(i = 0; i < n; i++) {
        if(m[i].id==id) {
            printf("Enter New Rating : ");
            scanf("%f", &rating);

            m[i].rating = rating;

            printf("Rating Updated Successfully!\n");
            return;
        }
    }

    printf("Content Not Found!\n");
}
```

```c
/*-----------------------------------------------
    Main Function
-----------------------------------------------*/
int main() {
    int choice;

    do {
        printf("\n\n");
        printf("========================================\n");
        printf("      MOVIE RECOMMENDATION SYSTEM\n");
        printf("========================================\n");

        printf("1. View All Content\n");
        printf("2. Search By Title\n");
        printf("3. Search By Genre\n");
        printf("4. Top Rated Content\n");
        printf("5. Recommend For Me\n");
        printf("6. Sort By Rating\n");
        printf("7. Rate Content\n");
        printf("8. Exit\n\n");

        printf("Enter Choice : ");
        scanf("%d",&choice);

        switch(choice) {
            case 1:
                displayAll();
                break;
            case 2:
                searchTitle();
                break;
            case 3:
                searchGenre();
                break;
            case 4:
                topRated();
                break;
            case 5:
                recommendMovie();
                break;
            case 6:
                sortRating();
                break;
            case 7:
                rateMovie();
                break;
            case 8:
                printf("Thank You!\n");
                break;
            default:
                printf("Invalid Choice!\n");
        }

    } while(choice != 8);

    return 0;
}
```

Output Screenshot:



Snake and Ladder in C

Problem Statement:

The objective of this project is to design and implement a **Snake and Ladder** game using the C programming language for two players.

Each player rolls a dice and moves across a 10x10 board (from 1 to 100). The game simulates the movement of players, snakes, and ladders, and determines the winner when a player reaches position **100**. The project helps understand **loops, conditionals, arrays, random number generation**, and **function-based modular programming**.

Concepts Used:

- **Loops:** To iterate through the board and run the game continuously.
- **Arrays:** For the 10x10 conceptual board layout.
- **Functions:** For modularity separate logic for dice roll, board display, and movement.
- **Conditionals (if-else):** To handle player turns, snakes, ladders, and game outcomes.
- **Random Number Generation (rand()):** To simulate dice rolls between 1 and 6.
- **System Functions (system("cls")):** To refresh and display updated game boards clearly.

Short Description of Working:

1. The program starts with both players at position **0**.
2. On each turn, a player presses a key to roll the dice (a random number from 1–6).
3. The player moves forward by that dice value.
4. If the player lands on a **ladder**, they move **up** to a higher position.
5. If the player lands on a **snake**, they slide **down** to a lower position.
6. The board updates dynamically after every move:
   - **'L'** marks ladder positions.
   - **'S'** marks snake positions.
   - **'P1'** and **'P2'** show the current player positions.
7. The first player to reach **exactly 100** wins the game.
8. The game runs in a loop until a winner is declared or the user exits.

snakeAndLadder.c:

```c
/*-------------------------------------------------------------
    Project Title   : Snake and Ladder Game
    Description     : A two-player console-based Snake and Ladder game
                      that simulates dice rolls, snakes, ladders, and
                      displays the board dynamically with player positions.
    Rules and Features:
     - The game begins with any dice value (1-6).
     - If the dice rolls 6, the player gets another chance.
     - Snakes and Ladders are shown as 'S' and 'L' on the board.
     - Players are displayed as 'P1' and 'P2' directly on the board.
     - Snakes: 99→1, 65→40, 25→9
       Ladders: 13→42, 60→83, 70→93
     - The first player to reach exactly 100 wins the game.
    Concepts Used   : Loops, Functions, Arrays, Conditional Statements,
                      Switch Case, Random Number Generation.
-------------------------------------------------------------*/



#include <stdio.h>
#include <stdlib.h>

/*-------------------------------------------------------------
   Function: rd
   Purpose : Generates a random dice value between 1 and 6.
-------------------------------------------------------------*/
int rd() {
    int rem;
A:
    rem = rand() % 7;
    if (rem == 0)
        goto A;
    else
        return rem;
}

/*-------------------------------------------------------------
   Function: displayChart
   Purpose : Displays the 10x10 Snake and Ladder board.
             Marks Snakes (S), Ladders (L), and Player Positions.
-------------------------------------------------------------*/
void displayChart(int p1, int p2) {
    int i, j, num, shift = 0;

    printf("\n\t\t\t---- SNAKE AND LADDER BOARD ----\n\n");

    // The board contains 10 rows (10x10 grid)
    for (i = 10; i > 0; i--) {
        int rowBase = (i - 1) * 10;

        // Left-to-right numbering
        if (shift % 2 == 0) {
            for (j = 10; j >= 1; j--) {
                num = rowBase + j;

                // Display player positions
                if (num == p1 && num == p2)
                    printf("P1P2*\t");
                else if (num == p1)
                    printf("P1*\t");
                else if (num == p2)
                    printf("P2*\t");

                // Display Snakes and Ladders
                else if (num == 99 || num == 65 || num == 42 || num == 25)
                    printf("S\t");
                else if (num == 79 || num == 64 || num == 36 || num == 13)
                    printf("L\t");
                else
                    printf("%d\t", num);
            }
        } else {  // Right-to-left numbering
            for (j = 1; j <= 10; j++) {
                num = rowBase + j;
```

```c
                if (num == p1 && num == p2)
                    printf("P1P2*\t");
                else if (num == p1)
                    printf("P1*\t");
                else if (num == p2)
                    printf("P2*\t");
                else if (num == 99 || num == 65 || num == 42 || num == 25)
                    printf("S\t");
                else if (num == 79 || num == 64 || num == 36 || num == 13)
                    printf("L\t");
                else
                    printf("%d\t", num);
            }
        }

        shift++;
        printf("\n\n");
    }

    printf("-------------------------------------------------------------------------\n");
    printf("Player 1 Position: %d\t|\tPlayer 2 Position: %d\n", p1, p2);
    printf("-------------------------------------------------------------------------\n");
}

/*-------------------------------------------------------------
   Function: movePlayer
   Purpose : Moves the player according to dice value and applies
             Snake or Ladder effect if present.
-------------------------------------------------------------*/
int movePlayer(int curPos, int dice, int playerNum) {
    curPos += dice;

    // Check if player exceeds 100
    if (curPos > 100) {
        curPos -= dice;
        printf("Player %d exceeded 100! Staying at %d.\n", playerNum, curPos);
        return curPos;
    }

    // Snake positions
    if
        (curPos == 99) curPos = 1;
    else if
        (curPos == 65) curPos = 40;
    else if
        (curPos == 25) curPos = 9;

    // Ladder positions
    else if
        (curPos == 13) curPos = 42;
    else if
        (curPos == 60) curPos = 83;
    else if
        (curPos == 70) curPos = 93;

    return curPos;
}

/*-------------------------------------------------------------
   Function: main
   Purpose : The main control function to execute the game.
             Handles player turns, dice rolls, and win conditions.
-------------------------------------------------------------*/
int main() {
    int curPos1 = 0, curPos2 = 0, dice;
    char choice;

    while (1) {
        system("cls"); // Clears the screen (Windows only)
        printf("\n\t\t\t    ** SNAKE AND LADDER GAME **\n\t\t\t      (Two Player Version)\n");
        printf("-------------------------------------------------------------------------\n");
```

```c

        displayChart(curPos1, curPos2);

        printf("\n1. Player 1 plays\n2. Player 2 plays\n3. Exit\nEnter your choice: ");
        scanf(" %c", &choice);

        switch (choice) {
            case '1':
                dice = rd();
                printf("\nPlayer 1 rolled a %d\n", dice);
                curPos1 = movePlayer(curPos1, dice, 1);

                if (curPos1 == 100) {
                    displayChart(curPos1, curPos2);
                    printf("\n Player 1 Wins the Game! \n");
                    exit(0);
                }
                break;

            case '2':
                dice = rd();
                printf("\nPlayer 2 rolled a %d\n", dice);
                curPos2 = movePlayer(curPos2, dice, 2);

                if (curPos2 == 100) {
                    displayChart(curPos1, curPos2);
                    printf("\n Player 2 Wins the Game! \n");
                    exit(0);
                }
                break;

            case '3':
                printf("Thanks for playing!\n");
                exit(0);

            default:
                printf("Invalid choice! Try again.\n");
        }

        printf("\nPress Enter to continue...");
        getchar();
        getchar();
    }

    return 0;
}
```

Output Screenshot: